LAB 7 – FUZZ TESTING

Thermostat Control System – Input Validation Testing

Student Name: Shwetha S
SRN: PES1UG24CS452

Course: Software Engineering
Experiment: Fuzz Testing using Hypothesis and Pytest
Repository: L6_L7_29_thermostat_control

1. Objective

The objective of this experiment is to perform fuzz testing on the Thermostat Control System using the Hypothesis library.

Fuzz testing automatically generates different inputs, including invalid and unexpected values, to identify input validation bugs and improve the reliability of the application.

2. Tools and Technologies Used

- Python 3.13
- Pytest
- Hypothesis
- Visual Studio Code

3. Functions Tested

The following three functions from the Thermostat Control System were tested:

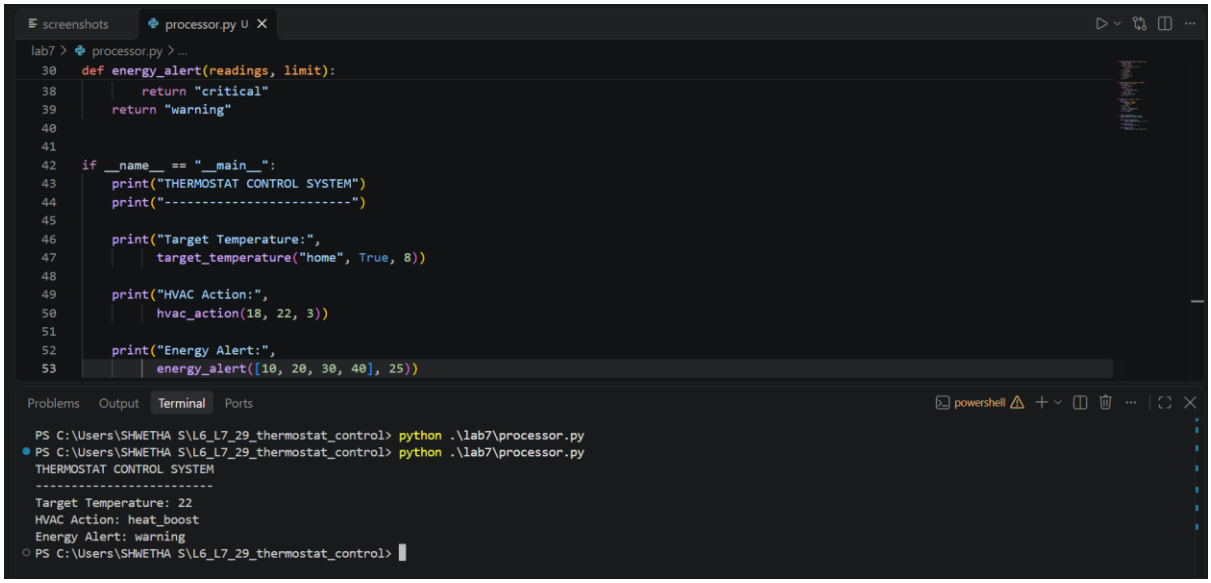
1. **target_temperature():** Determines the desired room temperature based on the operating mode, occupancy, and hour.
2. **hvac_action():** Determines whether the HVAC system should heat, cool, or remain idle.
3. **energy_alert():** Evaluates energy readings and generates an alert.

4. Initial Application Execution

The original application was executed before applying input validation.

The application produced the following output:

- Target Temperature: 22
- HVAC Action: heat_boost
- Energy Alert: warning



5. Fuzz Testing Implementation

Hypothesis was used with Pytest to automatically generate unexpected and invalid inputs.

The tests checked:

- Invalid operating modes.
- Invalid hour values.
- Invalid occupancy values.
- Invalid current, target, and outside temperatures.
- Invalid energy readings.
- Empty energy readings.
- Invalid energy limits.
- Valid energy readings.

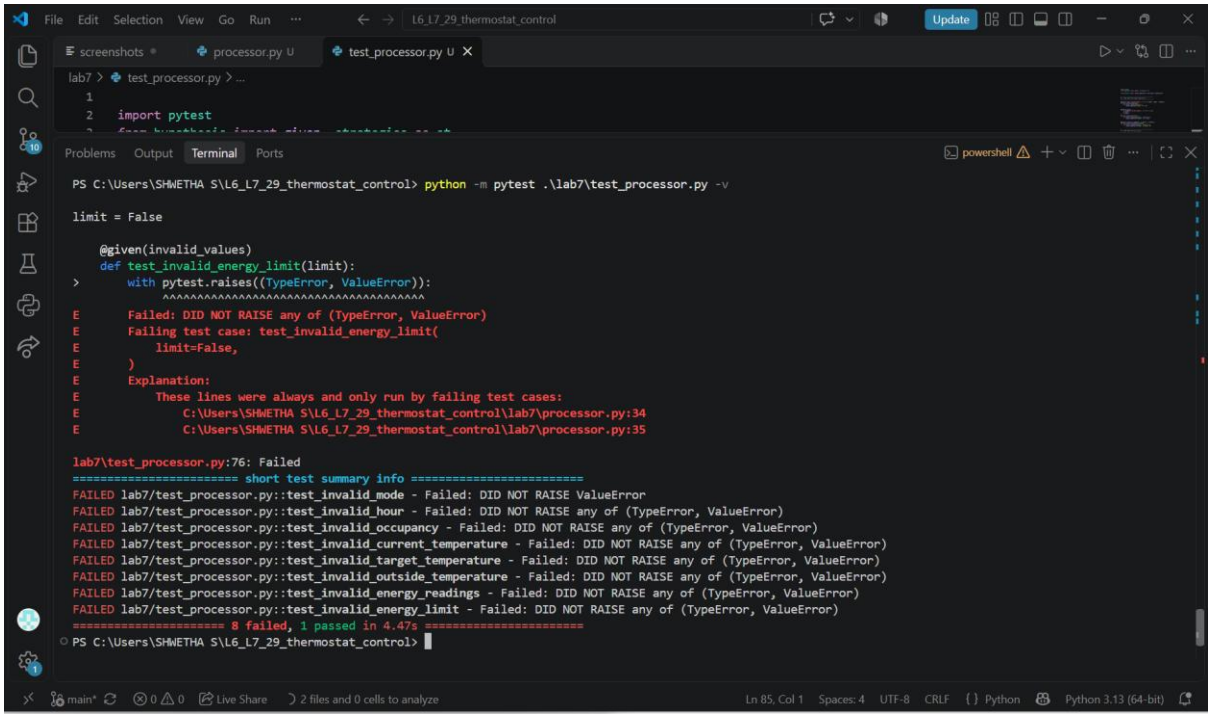
The tests used assertions and exception handling to check whether invalid inputs were rejected appropriately.

6. Bugs Identified

During the initial fuzz testing, 8 tests failed and 1 test passed.

The following issues were identified:

1. Invalid operating modes were accepted.
2. Invalid hour values were not rejected.
3. Invalid occupancy values were accepted.
4. Boolean values were incorrectly accepted as temperature inputs.
5. Invalid energy readings were not properly validated.
6. Empty energy readings were accepted.
7. Invalid energy limits were not properly validated.



7. Bug Fixes and Input Validation

The processor.py file was updated to include input validation.

The following improvements were implemented:

- Validated operating modes against the allowed values: home, away, and sleep.
- Restricted occupancy values to True or False.
- Restricted hour values to integers between 0 and 23.
- Validated temperature inputs to accept only finite numeric values.
- Rejected Boolean values where numeric temperatures were expected.
- Validated energy readings and energy limits.
- Rejected empty energy-reading collections.
- Added appropriate TypeError and ValueError exceptions.

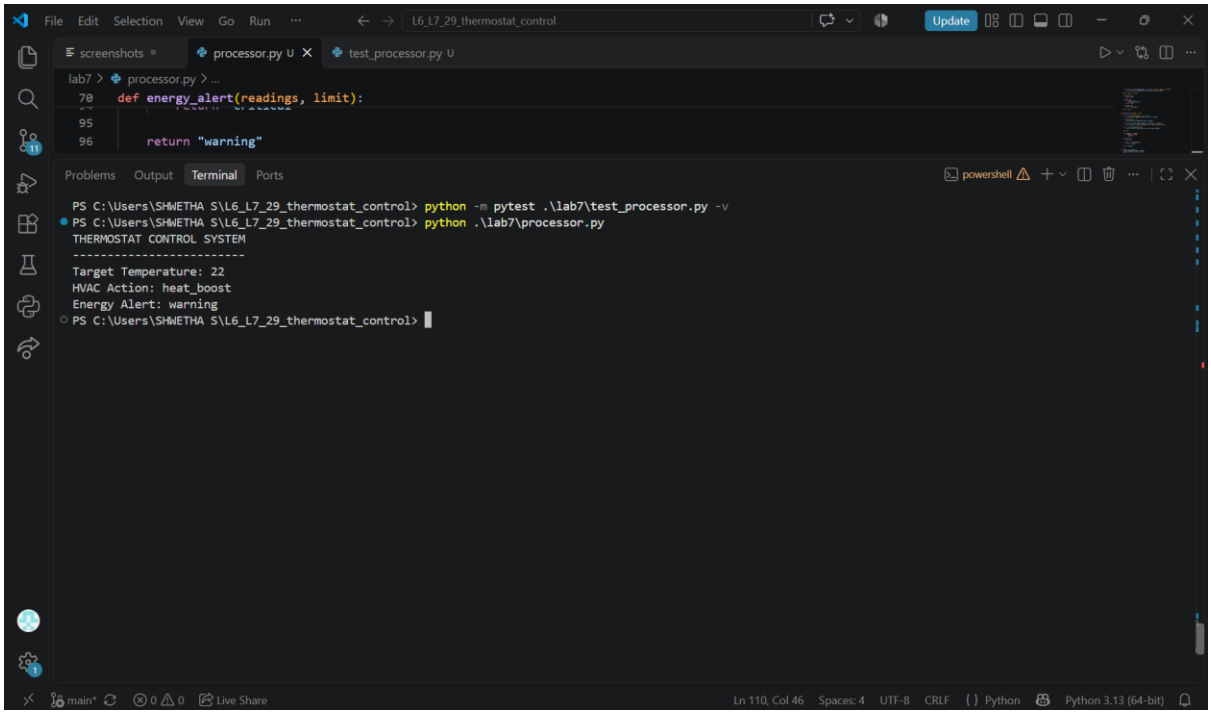
The original thermostat decision-making logic was preserved.

8. Fixed Application Execution

After implementing the validation logic, the application was executed again.

The application produced the expected output:

- Target Temperature: 22
- HVAC Action: heat_boost
- Energy Alert: warning



9. Final Fuzz Testing Results

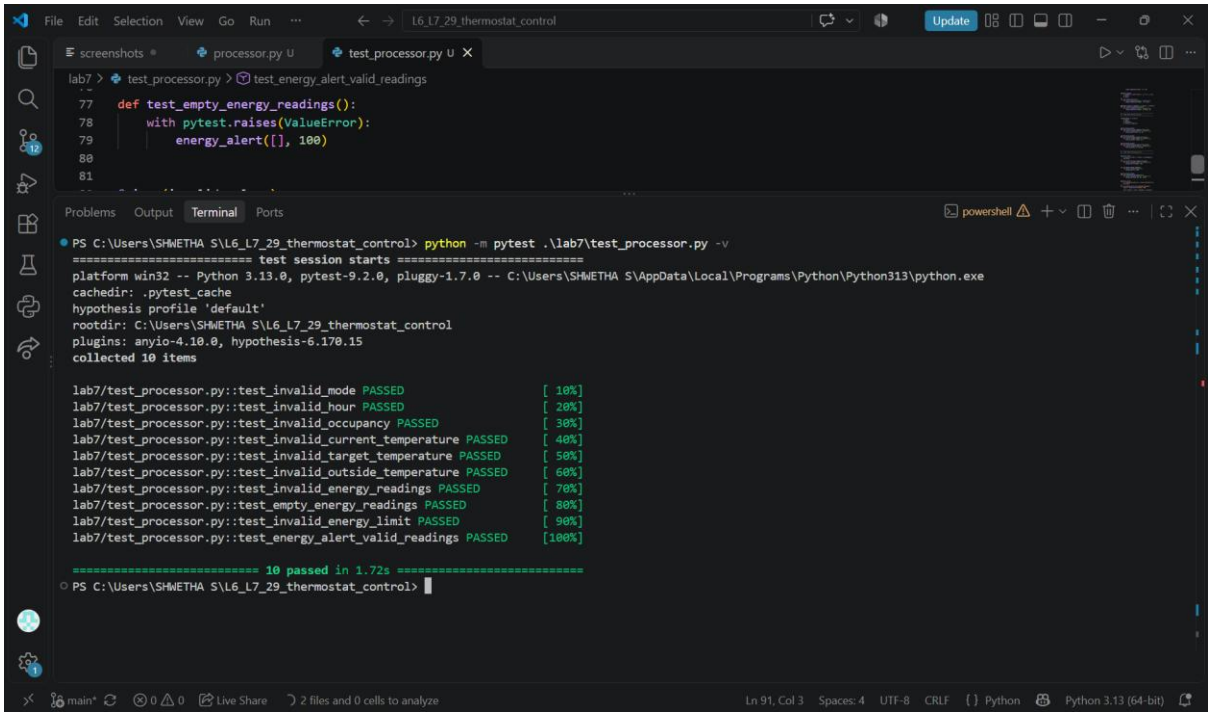
The updated application was tested again using Hypothesis and Pytest.

All 10 fuzz tests passed successfully.

Test	Result
Invalid mode	PASSED
Invalid hour	PASSED
Invalid occupancy	PASSED
Invalid current temperature	PASSED
Invalid target temperature	PASSED
Invalid outside temperature	PASSED
Invalid energy readings	PASSED
Empty energy readings	PASSED
Invalid energy limit	PASSED

Valid energy readings	PASSED

Final Result: 10 passed



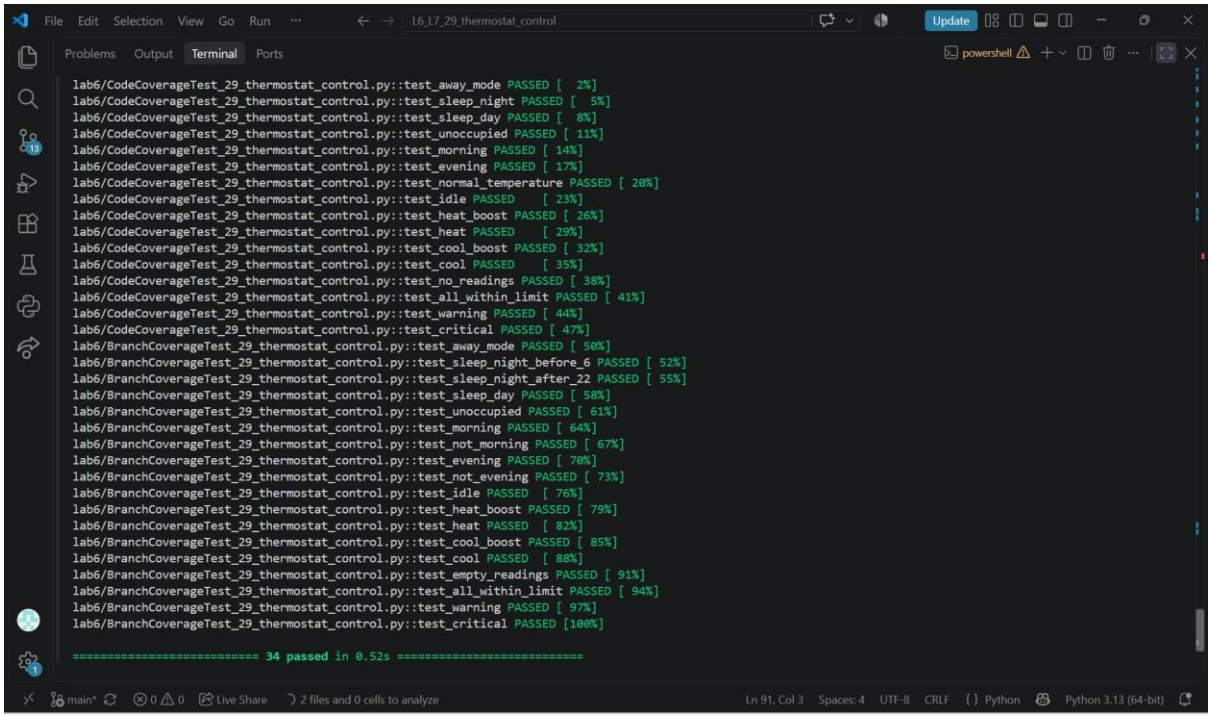
10. Regression Testing

Regression testing was performed to ensure that the Lab 7 changes did not affect the existing Lab 6 implementation.

The following results were obtained:

- Original Lab 6 tests: 2 passed.
- Code coverage tests: 16 passed.
- Branch coverage tests: 18 passed.

A total of 34 Lab 6 test executions passed across these test runs.



11. Conclusion

Fuzz testing was successfully performed on the Thermostat Control System using Hypothesis and Pytest.

The initial tests identified input validation weaknesses in the application. These issues were corrected by implementing appropriate input validation and exception handling.

After the modifications, all 10 Lab 7 fuzz tests passed successfully. Regression testing also confirmed that all 34 Lab 6 test executions passed across the test runs.

This experiment demonstrated how automated fuzz testing can identify unexpected input-handling issues and improve the robustness and reliability of software applications.